Machine Learning - Guida Completa allo Studio

Introduzione

Questa guida raccoglie e organizza in modo coerente tutti i concetti teorici e pratici trattati nei notebook di Machine Learning del repository. I materiali coprono un percorso completo che va dai fondamenti della regressione lineare fino alle tecniche avanzate di ensemble learning, ottimizzazione degli iperparametri e interpretazione dei modelli.

L'approccio didattico segue una progressione logica: dai modelli più semplici e interpretabili fino agli algoritmi più complessi, sempre con enfasi sulla comprensione concettuale, sulla corretta valutazione delle prestazioni e sulle best practice per lo sviluppo di pipeline ML robuste.

1. Fondamenti di Apprendimento Supervisionato

1.1 Il Paradigma Supervisionato

L'apprendimento supervisionato si basa su dati etichettati, dove ogni esempio di addestramento consiste in una coppia **(feature, target)**. L'obiettivo è imparare una funzione mappante che, date nuove feature, possa predire il target corrispondente.

Componenti fondamentali:

- **Feature (X)**: Le caratteristiche o variabili indipendenti che descrivono ogni esempio (es: superficie, numero di camere, reddito medio)
- **Target (y)**: La variabile dipendente che vogliamo predire (es: prezzo della casa, se un cliente abbandona il servizio)
- **Modello**: La funzione matematica che apprende la relazione tra X e y
- **Addestramento**: Il processo di ottimizzazione dei parametri del modello sui dati di training

1.2 Split dei Dati: Train/Test Split

Una pratica fondamentale è dividere i dati in due set distinti:

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # Il 20% dei dati per il test
    random_state=42,    # Riproducibilità dei risultati
    stratify=y          # Mantiene la distribuzione delle classi (per classificazione)
)
```

Perché dividere i dati?

- Il **training set** (solitamente 70-80%) viene usato per addestrare il modello
- Il **test set** (20-30%) viene usato per valutare le prestazioni su dati mai visti
- Senza questa divisione, non potremmo stimare quanto il modello generalizza

Stratified Split: Per problemi di classificazione con classi sbilanciate, è essenziale usare `stratify=y` per mantenere la stessa proporzione di classi sia nel training che nel test set.

Nota sul `random_state`: Il valore `42` è diventato un "seme delle risposte facili" perché spesso produce split particolarmente favorevoli. Per valutazioni più realistiche, è consigliabile provare diversi valori di `random_state` o usare tecniche di cross-validation.

2. Regressione Lineare

2.1 L'Equazione Fondamentale

La regressione lineare è il fondamento di tutti gli algoritmi di apprendimento supervisionato. Tecnicamente, cerca di modellare la relazione tra una variabile dipendente continua e una o più variabili indipendenti.

Equazione per un singolo esempio:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

Dove:

- **y**: La variabile dipendente (target, es: prezzo della casa)
- **β_0**: L'**intercetta** (valore di y quando tutte le feature sono zero)
- **$\beta_1, \ldots, \beta_n$**: I **coefficienti** (o pesi). Indicano di quanto varia y per ogni variazione unitaria della relativa feature
- **$x_1, \ldots, x_n$**: Le feature (variabili indipendenti)

- **ε**: Il **termine di errore** (residuo), rappresenta tutto ciò che il modello non riesce a spiegare

In notazione matriciale:

$$\hat{y} = X\beta$$

Dove X è la matrice delle feature e β è il vettore dei coefficienti.

2.2 La Funzione di Costo: OLS (Ordinary Least Squares)

Il modello apprende i coefficienti β minimizzando la **funzione di costo**, che misura l'errore tra predizioni e valori reali.

Mean Squared Error (MSE):

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Perché quadratizzare gli errori?

1. **Evita cancellazioni**: Gli errori positivi e negativi non si annullano
2. **Penalizza gli errori grandi**: Un errore di 2 conta 4 volte più di un errore di 1
3. **Proprietà matematiche**: La funzione è derivabile e convessa, facilitando l'ottimizzazione

RMSE (Root Mean Squared Error):

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

L'RMSE è preferibile al MSE perché è nella stessa unità di misura del target, rendendolo più interpretabile.

R^2 Score (Coefficient of Determination):

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

R^2 misura la proporzione di varianza nel target spiegata dal modello:

- **$R^2 = 1$**: Modello perfetto
- **$R^2 = 0$**: Il modello non è migliore della media
- **$R^2 < 0$**: Il modello è peggiore della media

2.3 Implementazione Pratica

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Creazione e addestramento del modello
model = LinearRegression()
model.fit(X_train, y_train)

# Predizione
y_pred = model.predict(X_test)

# Valutazione
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.4f}")
print(f"R²: {r2:.4f}")
```

Interpretazione dei coefficienti:

```python
# Intercetta
print(f"Intercept: {model.intercept_:.4f}")

# Coefficienti per feature
for feature, coef in zip(feature_names, model.coef_):
    print(f"{feature}: {coef:.4f}")
```

Ogni coefficiente rappresenta l'effetto marginale di quella feature sul target, mantenendo costanti le altre feature.

3. Gestione degli Outlier

3.1 Cos'è un Outlier

Un outlier è un'osservazione che si discosta significativamente dalle altre osservazioni nel dataset. Gli outlier possono:

- Distorcere le stime dei parametri
- Ridurre le prestazioni del modello
- Indicare errori di misurazione o dati corrotti

3.2 Metodo Z-Score

Il metodo Z-score standardizza i dati e identifica outlier come valori con Z-score assoluto maggiore di una soglia (tipicamente 3).

Formula Z-Score:

$$Z = \frac{x - \mu}{\sigma}$$

Dove:

- **x**: Il valore originale
- **μ**: La media della feature
- **σ**: La deviazione standard della feature

Interpretazione:

- **|Z| < 3**: Il valore è considerato normale (99.7% dei dati in distribuzione normale)
- **|Z| >= 3**: Il valore è considerato outlier

Implementazione:

```
import pandas as pd
import numpy as np

# Calcolo Z-score per tutte le feature
media = df[feature_cols].mean()
dev_std = df[feature_cols].std()

z = (df[feature_cols] - media) / dev_std

# Identificazione outlier (almeno una feature con |Z| >= 3)
mask = (z.abs() < 3).all(axis=1)

# Rimozione outlier
df_clean = df[mask].copy()
```

Considerazioni:

- Il metodo Z-score assume una distribuzione approssimativamente normale
- È sensibile alla presenza di outlier stessi (che distorcono media e deviazione standard)
- Per distribuzioni non normali, considerare il metodo IQR

3.3 Metodo IQR (Interquartile Range)

Il metodo IQR è più robusto agli outlier perché utilizza i quartili invece di media e deviazione standard.

Formula IQR:

$$IQR = Q_3 - Q_1$$

Dove:

- **Q_1**: Primo quartile (25° percentile)
- **Q_3**: Terzo quartile (75° percentile)

Soglie per outlier:

- **Limite inferiore**: $Q_1 - 1.5 \times IQR$
- **Limite superiore**: $Q_3 + 1.5 \times IQR$

Implementazione:

```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1

# Identificazione outlier
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

mask = (df >= lower_bound) & (df <= upper_bound)
df_clean = df[mask.all(axis=1)]
```

Confronto Z-Score vs IQR:

- **Z-Score**: Migliore per distribuzioni normali, più aggressivo
- **IQR**: Più robusto, funziona meglio con distribuzioni skewate

3.4 Quando Rimuovere gli Outlier

Criteri per la rimozione:

- **< 5% di missing values**: Rimuovi le righe con outlier
- **< 15-20% di missing values**: Sostituisci con media/mediana
- **> 20% di missing values**: Elimina la colonna o predici con ML

Importante: Rimuovere gli outlier deve essere giustificato dal dominio. A volte gli outlier rappresentano casi legittimi ma rari che il modello dovrebbe imparare a gestire.

4. Classificazione

4.1 Differenza tra Regressione e Classificazione

La **classificazione** è un compito di apprendimento supervisionato dove il target è **categorico** (discreto), non continuo.

Classificazione binaria: Due classi (es: 0/1, Sì/No, Spam/Non Spam)

Classificazione multiclasse: Più di due classi (es: Iris setosa/versicolor/virginica)

4.2 Logistic Regression

Nonostante il nome, la Logistic Regression è un algoritmo di classificazione, non regressione.

Concetto Fondamentale:

Mentre la regressione lineare predice valori continui, la logistic regression applica una **funzione sigmoide** che "schiaccia" il risultato tra 0 e 1, interpretandolo come probabilità.

Funzione Sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Dove $z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$

Decision Boundary:

- **0 ≤ p < 0.5**: Classe 0
- **0.5 ≤ p ≤ 1**: Classe 1

La soglia di 0.5 può essere adattata in base al dominio (es: in medicina, per rilevare malattie, potremmo abbassare la soglia per essere più conservativi).

Implementazione:

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Creazione e addestramento
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# Predizione
predictions = model.predict(X_test)

# Valutazione
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
```

4.3 Interpretazione della Sigmoide

La sigmoide trasforma una combinazione lineare in una probabilità:

- Valori molto negativi → probabilità vicina a 0
- Valori attorno a 0 → probabilità vicina a 0.5 (incertezza massima)
- Valori molto positivi → probabilità vicina a 1

Esempio:

- Se la sigmoide restituisce 0.3 → il modello è "30% sicuro" che appartenga alla classe 1
- Se restituisce 0.8 → il modello è "80% sicuro" che appartenga alla classe 1

5. Valutazione dei Modelli di Classificazione

5.1 Matrice di Confusione

La matrice di confusione è uno strumento fondamentale per capire **dove** il modello sbaglia, non solo **quanto** sbaglia.

Struttura per classificazione binaria:

	Predetto Negativo	Predetto Positivo
Reale Negativo	TN (True Negative)	FP (False Positive)
Reale Positivo	FN (False Negative)	TP (True Positive)

Definizioni:

- **TP (True Positive)**: Positivi correttamente predetti come positivi
- **TN (True Negative)**: Negativi correttamente predetti come negativi
- **FP (False Positive)**: Negativi predetti erroneamente come positivi (Errore Tipo I)
- **FN (False Negative)**: Positivi predetti erroneamente come negativi (Errore Tipo II)

Implementazione:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import seaborn as sns
import matplotlib.pyplot as plt

# Matrice di confusione
cm = confusion_matrix(y_test, predictions)

# Visualizzazione
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title('Matrice di Confusione')
plt.show()
```

Interpretazione:

- Una matrice ben bilanciata ha valori alti sulla diagonale principale
- Pattern fuori diagonale indicano dove il modello si confonde
- Utile per identificare classi problematiche che richiedono più dati o feature migliori

5.2 Metriche Derivate dalla Matrice di Confusione

Accuracy (Accuratezza):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Limiti dell'Accuracy: In dataset sbilanciati, un modello che predice sempre la classe maggioritaria può avere accuracy alta ma essere inutile.

Precision (Precisione):

$$\text{Precision} = \frac{TP}{TP + FP}$$

Misura: "Dei casi predetti positivi, quanti sono realmente positivi?"

Recall (Sensibilità):

$$\text{Recall} = \frac{TP}{TP + FN}$$

Misura: "Dei casi realmente positivi, quanti ne ho catturati?"

F1-Score:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean di Precision e Recall. Utile quando si vuole bilanciare le due metriche.

ROC AUC (Receiver Operating Characteristic - Area Under Curve):

Misura la capacità del modello di distinguere tra le classi a diverse soglie di decisione.

- **AUC = 1**: Modello perfetto
- **AUC = 0.5**: Modello casuale (come indovinare)
- **AUC < 0.5**: Modello peggiore del caso (invertire le predizioni)

Implementazione completa:

```
from sklearn.metrics import classification_report, roc_auc_score

# Report completo
print(classification_report(y_test, predictions))

# ROC AUC
auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
print(f"ROC AUC: {auc:.4f}")
```

6. Analisi Esplorativa dei Dati (EDA)

6.1 Statistiche Descrittive

```
# Statistiche di base
df.describe()  # media, std, min, max, quartili per colonne numeriche

# Informazioni sulle colonne
df.info()  # tipi di dati, missing values, uso memoria

# Distribuzione delle classi
df['target'].value_counts()
df['target'].value_counts(normalize=True)  # percentuali
```

6.2 Gestione dei Missing Values

Strategie di gestione:

1. **< 5% di missing**: Rimuovi le righe con valori mancanti
2. **< 15-20% di missing**: Sostituisci con:
   - **Media** per distribuzioni normali
   - **Mediana** per distribuzioni skewate (più robusta)
   - **Moda** per variabili categoriche
3. **> 20% di missing**:
   - Elimina la colonna
   - Oppure usa ML per predire i valori mancanti (es: KNN Imputer)

Implementazione:

```
# Identificazione missing values
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)) * 100

# Imputazione
df['column'] = df['column'].fillna(df['column'].median())  # Numerica
df['column'] = df['column'].fillna(df['column'].mode()[0])  # Categorica

# Gestione categoriche con "Unknown"
for col in categorical_cols:
    df[col] = df[col].cat.add_categories(["Unknown"])
    df[col] = df[col].fillna("Unknown")
```

6.3 Visualizzazioni Essenziali

Pairplot: Visualizza le relazioni tra tutte le coppie di feature

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(df, hue='target', diag_kind='hist')
plt.show()
```

Matrice di Correlazione:

```
# Correlazione di Pearson (lineare)
corr_matrix = df.corr(method='pearson')

# Correlazione di Spearman (non lineare)
corr_spearman = df.corr(method='spearman')

# Visualizzazione
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Matrice di Correlazione')
plt.show()
```

Importante: **Correlazione non è causalità**. Due feature possono essere correlate senza che una causi l'altra.

6.4 Distribuzioni Skewate

Skewness (Asimmetria):

- **Skew > 0**: Distribuzione spostata a destra (coda lunga a destra)
- **Skew < 0**: Distribuzione spostata a sinistra (coda lunga a sinistra)
- **Skew ≈ 0**: Distribuzione simmetrica

Trasformazione Logaritmica:

Per ridurre la skewness e rendere le distribuzioni più simili a una gaussiana:

```
# Trasformazione logaritmica (log(1+x) per gestire zeri)
df['column_log'] = np.log1p(df['column'])

# Verifica
print(f"Skew prima: {df['column'].skew():.2f}")
print(f"Skew dopo: {df['column_log'].skew():.2f}")
```

Altre trasformazioni:

- **Log**: Per dati positivi con skewness positiva
- **Square root**: Trasformazione più leggera del log
- **Box-Cox**: Per dati positivi (richiede dati > 0)

7. Feature Engineering

7.1 Cos'è il Feature Engineering

Il feature engineering è il processo di creazione, selezione e trasformazione delle feature per migliorare le prestazioni del modello. Spesso è più importante della scelta dell'algoritmo stesso.

7.2 Creazione di Feature

Feature di combinazione:

```
# Esempio Titanic: dimensione della famiglia
df['family_size'] = df['sibsp'] + df['parch'] + 1

# Esempio Titanic: costo per persona
df['fare_per_person'] = df['fare'] / df['family_size']

# Esempio Kaggle Telco: spesa mensile media
df['Avg_Monthly_Spend'] = df['TotalCharges'] / (df['tenure'] + 1)

# Feature temporali
df['Tenure_Years'] = df['tenure'] / 12
```

Feature booleane:

```
# Esempio Titanic: passeggero solo
df['is_alone'] = (df['family_size'] == 1).astype(int)

# Esempio Spaceship Titanic: nessun spesa
df['NoSpending'] = (df['TotalSpending'] == 0).astype('category')
```

Feature ratio:

```
# Esempio Penguins: ratio tra caratteristiche
df['bill_ratio'] = df['bill_length_mm'] / df['bill_depth_mm']
df['mass_flipper_ratio'] = df['body_mass_g'] / df['flipper_length_mm']
```

Feature squared (non linearità):

```
df['age_squared'] = df['age'] ** 2
```

7.3 Binning (Discretizzazione)

Dividere una feature continua in categorie basate su percentili:

```
# Qcut: divisione in intervalli con uguale numero di campioni
df['body_mass_bin'] = pd.qcut(
    df['body_mass_g'],
    q=3,  # 3 categorie
    labels=['Small', 'Medium', 'Large'],
    duplicates='drop'  # Gestisci casi di valori duplicati
)

# Cut: divisione in intervalli fissi
df['age_group'] = pd.cut(
    df['age'],
    bins=[0, 18, 35, 60, 100],
    labels=['Child', 'Young', 'Adult', 'Senior']
)
```

7.4 Feature Categorical

Label Encoding: Assegna un numero intero a ogni categoria

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['encoded'] = le.fit_transform(df['categorical_column'])
```

One-Hot Encoding: Crea una colonna binaria per ogni categoria

```
# Con pandas
df_encoded = pd.get_dummies(df, columns=['categorical_column'], drop_first=True)

# Con sklearn (preferibile per pipeline)
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop='first', sparse_output=False)
ohe.fit(X_train[['categorical_column']])
X_train_encoded = ohe.transform(X_train[['categorical_column']])
X_test_encoded = ohe.transform(X_test[['categorical_column']])
```

Importante: Quando si usa One-Hot Encoding, è essenziale applicare lo stesso encoding al test set che al training set. Se il test set ha categorie non viste nel training, si verifica un errore.

Categorical Native (XGBoost):
XGBoost supporta nativamente feature categorical con `enable_categorical=True`, evitando la necessità di One-Hot Encoding esplicito.

```
model = xgb.XGBClassifier(
    tree_method='hist',
    enable_categorical=True,  # Supporto nativo per categorical
    n_estimators=100,
    max_depth=4
)
```

7.5 Normalizzazione e Standardizzazione

Perché normalizzare?

- Evitare che feature con valori enormi dominino quelle con valori piccoli
- Molti algoritmi (SVM, reti neurali, gradient descent) richiedono feature scalate

MinMaxScaler (Normalizzazione):

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Riduce i valori nell'intervallo [0, 1].

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # Usa gli stessi min/max del training
```

StandardScaler (Standardizzazione):

$$z = \frac{x - \mu}{\sigma}$$

Rende i dati con media 0 e deviazione standard 1.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Regola fondamentale: **Sempre fit sul training set, trasformare sia training che test set**. Mai fit sul test set o su tutto il dataset insieme.

8. Ensemble Learning

8.1 Concetto Fondamentale

L'ensemble learning combina più modelli per ottenere prestazioni migliori rispetto a un singolo modello. Il principio è la "wisdom of the crowd": un gruppo di esperti mediocri spesso batte un singolo esperto.

8.2 Bagging (Bootstrap Aggregating)

Random Forest è l'algoritmo di bagging più popolare.

Principio:

1. Crea molti bootstrap samples (campioni con replacement dal dataset originale)
2. Addestra un modello su ogni sample
3. Combina le predizioni (media per regressione, voto per classificazione)

Perché funziona?

- Riduce la varianza (overfitting)
- Ogni modello vede una versione leggermente diversa dei dati
- Gli errori si compensano a vicenda

Implementazione:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,      # Numero di alberi
    max_depth=None,        # Profondità massima (None = illimitata)
    min_samples_split=2,   # Minimi campioni per dividere un nodo
    min_samples_leaf=1,    # Minimi campioni in una foglia
    max_features='sqrt',   # Feature considerate per split
    random_state=42,
    n_jobs=-1              # Usa tutti i core CPU
)
```

Iperparametri chiave:

- **n_estimators**: Più alberi = migliore prestazioni (fino a un punto di saturazione)
- **max_depth**: Controlla l'overfitting (profondità limitata = meno overfitting)
- **min_samples_split/min_samples_leaf**: Regola la complessità dell'albero
- **max_features**: Numero di feature considerate per ogni split

8.3 Boosting

A differenza del bagging (modelli paralleli e indipendenti), il boosting addestra modelli **sequenzialmente**, dove ogni modello cerca di correggere gli errori del precedente.

XGBoost (eXtreme Gradient Boosting) è l'implementazione più popolare.

Principio:

1. Inizia con un modello semplice
2. Calcola gli errori (residuali)
3. Addestra un nuovo modello sui residuali
4. Combina i modelli con un learning rate

Implementazione:

```python
import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=500,        # Numero di alberi
    max_depth=6,             # Profondità massima (tipicamente 3-10)
    learning_rate=0.1,       # Contributo di ogni albero (tipicamente 0.01-0.3)
    subsample=0.8,           # Frazione di campioni per ogni albero
    colsample_bytree=0.8,    # Frazione di feature per ogni albero
    gamma=0,                 # Regularizzazione minima perdita (0 = disabilitato)
    reg_alpha=0,             # L1 regularization
    reg_lambda=1,            # L2 regularization
    scale_pos_weight=1,      # Per classi sbilanciate
    tree_method='hist',      # Algoritmo efficiente per grandi dataset
    random_state=42,
    use_label_encoder=False  # Non deprecato
)
```

Iperparametri chiave:

- **n_estimators**: Numero di alberi (più = migliore, ma più lento)
- **max_depth**: Complessità di ogni albero (tipicamente 3-10)
- **learning_rate**: Velocità di apprendimento (più basso = più iterazioni necessarie)
- **subsample**: Diversità dei campioni (0.6-1.0)
- **colsample_bytree**: Diversità delle feature (0.6-1.0)
- **gamma**: Regularizzazione minima (0 = nessuna)
- **reg_alpha**: L1 regularization (contro overfitting)
- **reg_lambda**: L2 regularization (contro overfitting)
- **scale_pos_weight**: Per dataset sbilanciati (ratio classi)

Per problemi sbilanciati:

```python
# Calcola il ratio delle classi
ratio = len(y_train[y_train==0]) / len(y_train[y_train==1])

# Imposta scale_pos_weight
param = {
    'scale_pos_weight': ratio,
    'objective': 'binary:logistic',
    'eval_metric': 'logloss'
}
```

8.4 Confronto Bagging vs Boosting

Caratteristica	Bagging (Random Forest)	Boosting (XGBoost)
Addestramento	Parallelo	Sequenziale
Bias	Basso	Molto basso
Variance	Molto basso	Basso
Overfitting	Raro	Possibile (se troppi alberi)
Velocità	Veloce (parallelo)	Più lento (sequenziale)
Robustezza	Molto robusto	Sensibile a outlier

9. Support Vector Machines (SVM)

9.1 Concetto Fondamentale

Le SVM cercano di trovare il **margine massimo** che separa le classi. Il margine è la distanza tra il decision boundary e i punti più vicini di ciascuna classe (support vectors).

Perché margine massimo?

- Margini più ampi generalizzano meglio
- Meno sensibili al rumore nei dati

9.2 Kernel Trick

Per problemi non linearmente separabili, le SVM usano la **kernel trick** per mappare i dati in uno spazio di dimensione superiore dove sono linearmente separabili.

Kernel comuni:

- **Linear**: Per dati linearmente separabili
- **RBF (Radial Basis Function)**: Il più comune, gestisce non-linearità complesse
- **Polynomial**: Per relazioni polinomiali
- **Sigmoid**: Simile a una rete neurale

Implementazione:

```python
from sklearn.svm import SVC

# SVM con kernel RBF
model = SVC(
    kernel='rbf',        # Kernel Radial Basis Function
    C=1.0,               # Parametro di regolarizzazione
    gamma='scale',       # Inverso della regione influenzata da un singolo campione
    probability=True     # Per ottenere probabilità (più lento)
)
```

Iperparametri chiave:

- **C**: Parametro di regolarizzazione (alto = meno regolarizzazione, più overfitting)
- **gamma**: Influenza di ogni campione (alto = decision boundary più complesso)

10. Decision Trees

10.1 Concetto Fondamentale

I Decision Trees creano un modello ad albero che prende decisioni sequenziali basate sulle feature.

Struttura:

- **Root node**: Il primo split (feature più informativa)
- **Internal nodes**: Split aggiuntivi
- **Leaves**: Predizioni finali

10.2 Criteri di Split

Gini Impurity:

$$Gini = 1 - \sum_{i=1}^{C} p_i^2$$

Dove p_i è la proporzione di campioni della classe i-esima.

Entropy e Information Gain:

$$Entropy = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

$$InformationGain = Entropy(parent) - \sum_{i=1}^{k} \frac{n_i}{n} \times Entropy(child_i)$$

10.3 Implementazione

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion='gini',      # 'gini' o 'entropy'
    max_depth=10,          # Profondità massima
    min_samples_split=2,   # Minimi campioni per split
    min_samples_leaf=1,    # Minimi campioni in una leaf
    random_state=42
)
```

Problemi dei Decision Trees:

- Tendono a overfitting (specialmente se profondi)
- Instabili (piccole variazioni nei dati cambiano la struttura)
- **Bagging (Random Forest)** e **Boosting** risolvono questi problemi

11. Cross-Validation

11.1 Perché Cross-Validation?

Il train/test split semplice ha limiti:

- La valutazione dipende dallo split specifico
- Perde dati per il training (solo 80% usato)
- Valutazione instabile su dataset piccoli

11.2 K-Fold Cross-Validation

Procedura:

1. Dividi i dati in K fold (tipicamente 5 o 10)

2. Per ogni fold:
    - Usa 1 fold come test set
    - Usa K-1 fold come training set
    - Addestra e valuta il modello
3. Media le prestazioni su tutti i fold

Implementazione:

```
from sklearn.model_selection import cross_val_score, KFold

# K-Fold con 5 fold
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Valutazione con cross-validation
scores = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')

print(f"Accuracy media: {scores.mean():.4f} (+/- {scores.std():.4f})")
```

Stratified K-Fold: Mantiene la proporzione di classi in ogni fold (essenziale per classificazione sbilanciata).

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='accuracy')
```

12. Hyperparameter Tuning

12.1 Grid Search

Prova tutte le combinazioni di iperparametri specificate.

Implementazione:

```python
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

print(f"Best params: {grid_search.best_params_}")
print(f"Best score: {grid_search.best_score_:.4f}")
```

Pro: Esplora sistematicamente lo spazio
Contro: Computazionalmente costoso per spazi grandi

12.2 Random Search

Campiona casualmente combinazioni di iperparametri.

Implementazione:

```python
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

param_distributions = {
    'n_estimators': randint(100, 500),
    'max_depth': [3, 5, 10, 15, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'max_features': ['sqrt', 'log2', None]
}

random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions,
    n_iter=50,           # Numero di combinazioni da provare
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42
)

random_search.fit(X_train, y_train)
```

Pro: Più efficiente di Grid Search, trova buone combinazioni con meno iterazioni
Contro: Non garantisce di trovare il miglior punto

12.3 Optuna (Bayesian Optimization)

Optuna usa ottimizzazione bayesiana per guidare la ricerca verso regioni promettenti dello spazio degli iperparametri.

Vantaggi:

- Più efficiente di Grid/Random Search
- Supporta pruning (interrompe trial promettenti)
- Interfaccia intuitiva
- Visualizzazione integrata

Implementazione:

```python
import optuna
from optuna.integration import XGBoostPruningCallback
import xgboost as xgb

def objective(trial):
    # Definizione iperparametri da ottimizzare
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 200, 1200),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 20),
        'gamma': trial.suggest_float('gamma', 0.0, 5.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 0.0, 5.0),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.0, 10.0),
        'scale_pos_weight': len(y_train[y_train==0]) / len(y_train[y_train==1]),
        'objective': 'binary:logistic',
        'eval_metric': 'logloss',
        'tree_method': 'hist',
        'random_state': 42
    }

    # Cross-validation integrata
    dtrain = xgb.DMatrix(X_train, label=y_train, enable_categorical=True)
    dvalid = xgb.DMatrix(X_test, label=y_test, enable_categorical=True)

    model = xgb.train(
        params,
        dtrain,
        num_boost_round=1000,
        evals=[(dvalid, 'valid')],
        early_stopping_rounds=50,
        callbacks=[XGBoostPruningCallback(trial, 'validation-logloss')]
    )

    # Valutazione
    y_pred = model.predict(dvalid)
    from sklearn.metrics import roc_auc_score
    auc = roc_auc_score(y_test, y_pred)

    return auc  # Valore da massimizzare

# Ottimizzazione
study = optuna.create_study(direction='maximize')
```

```
study.optimize(objective, n_trials=100, timeout=3600)

print(f"Best trial: {study.best_trial.number}")
print(f"Best score: {study.best_value:.4f}")
print(f"Best params: {study.best_params}")
```

Suggest methods:

- `suggest_int(name, low, high)` : Intero in intervallo [low, high]
- `suggest_float(name, low, high)` : Float in intervallo [low, high]
- `suggest_float(name, low, high, log=True)` : Float in spazio logaritmico
- `suggest_categorical(name, choices)` : Scelta tra categorie

13. Interpretabilità dei Modelli

13.1 Feature Importance

Per alberi e ensemble:

```
# Feature importance standard
importances = model.feature_importances_
indices = np.argsort(importances)[::-1]

# Visualizzazione
plt.figure(figsize=(10, 6))
plt.bar(range(len(importances)), importances[indices])
plt.xticks(range(len(importances)), [feature_names[i] for i in indices], rotation=90)
plt.title('Feature Importance')
plt.tight_layout()
plt.show()
```

Interpretazione:

- Mostra quanto ogni feature contribuisce alle predizioni
- Non indica la direzione dell'effetto (positivo/negativo)
- Può essere fuorviante per feature correlate

13.2 SHAP (SHapley Additive exPlanations)

SHAP è basato sulla teoria dei giochi e fornisce spiegazioni consistenti e localmente accurate.

Concetto:

- Ogni feature è un "giocatore"
- Il valore SHAP misura il contributo marginale di ogni feature
- Somma dei valori SHAP + valore base = predizione finale

Implementazione:

```
import shap

# Creazione explainer
explainer = shap.Explainer(model, X_train)
shap_values = explainer(X_test)

# Summary plot (globale)
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

# Force plot (individuale)
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0])

# Dependence plot (relazione feature-target)
shap.dependence_plot("MedInc", shap_values, X_test)
```

Vantaggi di SHAP:

- Spiegazioni locali (per ogni predizione) e globali
- Consistenza teorica (teoria dei giochi)
- Visualizzazioni intuitive

13.3 LIME (Local Interpretable Model-agnostic Explanations)

LIME approssima localmente il modello complesso con un modello interpretabile (es: regressione lineare).

```python
import lime
import lime.lime_tabular

explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=feature_names,
    class_names=['Classe 0', 'Classe 1'],
    discretize_continuous=True
)

exp = explainer.explain_instance(
    X_test.iloc[0].values,
    model.predict_proba,
    num_features=5
)

exp.show_in_notebook()
```

14. Reti Neurali (Introduzione)

14.1 Concetto Fondamentale

Le reti neurali sono modelli ispirati al cervello biologico, composti da:

- **Neuroni (nodi)**: Unità computazionali base
- **Pesaggi (weights)**: Connessioni tra neuroni
- **Bias**: Offset per ogni neurone
- **Funzioni di attivazione**: Introducono non-linearità

14.2 Forward Pass

Il **forward pass** è il processo di propagazione degli input attraverso la rete fino all'output.

Struttura tipica:

1. Input layer: Riceve le feature
2. Hidden layers: Elaborazione con attivazioni non lineari
3. Output layer: Predizione finale

Esempio con Keras:

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=[n_features]),
    layers.Dropout(0.3),  # Regularizzazione
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(16, activation='relu'),
    layers.Dense(2, activation='softmax')  # Per classificazione binaria/multiclasse
])
```

14.3 Backpropagation

Il **backpropagation** è l'algoritmo di apprendimento che aggiorna i pesi per minimizzare la funzione di costo.

Analogia didattica:

> "Il feedback dell'allenatore che torna indietro lungo la fila"

Procedura:

1. Forward pass: Calcola le predizioni
2. Calcola la perdita (loss)
3. Backward pass: Calcola i gradienti (quanto ogni peso contribuisce all'errore)
4. Aggiorna i pesi con gradient descent

14.4 Dropout

Dropout è una tecnica di regularizzazione che previene l'overfitting.

Concetto:

- Durante ogni step di training, "spegne" casualmente una percentuale di neuroni
- Forza la rete a non dipendere troppo da singoli neuroni
- Ad inference time, tutti i neuroni sono attivi

Implementazione:

```
layers.Dropout(0.5)  # Spegne il 50% dei neuroni durante il training
```

Perché funziona:

- Ogni training step usa una "rete diversa" (sotto-rete)
- Effetto simile a ensemble di molte reti
- Riduce la co-adattamento dei neuroni

15. Overfitting e Regularizzazione

15.1 Cos'è l'Overfitting

L'**overfitting** (sovradattamento) si verifica quando il modello:

- Impara i pattern generali dei dati
- **MA** anche il rumore e le fluttuazioni casuali
- Presta bene sul training set, male sul test set

Segnali di overfitting:

- Accuracy training alta, accuracy test bassa
- Gap crescente tra training e validation loss
- Modello troppo complesso per la quantità di dati

15.2 Tecniche di Regularizzazione

Per Reti Neurali:

- **Dropout**: Spegne neuroni casualmente durante training
- **L1/L2 regularization**: Penalizza pesi grandi
- **Early stopping**: Ferma training quando validation loss aumenta
- **Data augmentation**: Aumenta dati con trasformazioni

Per Alberi/Ensemble:

- **Limit max_depth**: Limita profondità dell'albero
- **Min samples split/leaf**: Richiede più campioni per split
- **Max features**: Limita feature considerate per split
- **Subsample/colsample_bytree**: Randomizzazione nei boosting

Implementazione XGBoost:

```
model = xgb.XGBClassifier(
    max_depth=6,              # Limite profondità
    min_child_weight=5,       # Minimi campioni in una leaf
    subsample=0.8,            # Campioni per albero
    colsample_bytree=0.8,     # Feature per albero
    reg_alpha=0.1,            # L1 regularization
    reg_lambda=1.0,           # L2 regularization
    gamma=0.1                 # Perda minima per split
)
```

15.3 Underfitting

L'**underfitting** (sottodattamento) è il problema opposto:

- Modello troppo semplice per i dati
- Presta male sia su training che su test
- Bias alto, variance bassa

Soluzioni:

- Aumenta complessità del modello
- Aggiungi feature
- Riduci regularizzazione
- Addestra più a lungo (per reti neurali)

16. Pipeline ML Completa

16.1 Struttura di una Pipeline

Una pipeline ML ben strutturata include:

1. **Data Loading e Exploratory Analysis**
2. **Data Cleaning** (missing values, outlier detection)
3. **Feature Engineering** (creazione, trasformazione)
4. **Train/Test Split**
5. **Scaling/Normalization**
6. **Model Training**
7. **Hyperparameter Tuning**
8. **Model Evaluation**

9. **Interpretability** (feature importance, SHAP)
10. **Deployment** (saving model, creating submission)

16.2 Esempio End-to-End

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, roc_auc_score
import xgboost as xgb

# 1. Caricamento dati
df = pd.read_csv('data.csv')

# 2. Data Cleaning
# Gestione missing values
df['age'] = df['age'].fillna(df['age'].median())
df['categorical'] = df['categorical'].fillna('Unknown')

# Rimozione outlier (Z-score)
z = np.abs((df['numeric_feature'] - df['numeric_feature'].mean()) / df['numeric_feature'].st
df = df[z < 3]

# 3. Feature Engineering
df['ratio'] = df['feature1'] / df['feature2']
df['log_feature'] = np.log1p(df['feature3'])

# 4. Train/Test Split
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 5. Preprocessing
numeric_features = ['feature1', 'feature2', 'feature3']
categorical_features = ['cat1', 'cat2']

numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
```

```python
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)

# 6. Modello + Pipeline
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', xgb.XGBClassifier(
        n_estimators=500,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        tree_method='hist',
        random_state=42
    ))
])

# 7. Hyperparameter Tuning
param_grid = {
    'classifier__n_estimators': [300, 500, 700],
    'classifier__max_depth': [5, 6, 7],
    'classifier__learning_rate': [0.05, 0.1, 0.15]
}

grid_search = GridSearchCV(
    model, param_grid, cv=5, scoring='roc_auc', n_jobs=-1
)

grid_search.fit(X_train, y_train)

# 8. Valutazione
y_pred = grid_search.best_estimator_.predict(X_test)
y_proba = grid_search.best_estimator_.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(f"ROC AUC: {roc_auc_score(y_test, y_proba):.4f}")

# 9. Salva modello
import joblib
joblib.dump(grid_search.best_estimator_, 'model.pkl')
```

17. Competizioni Kaggle

17.1 Preparazione Submission

Struttura tipica:

```
import pandas as pd
import joblib

# Caricamento modello
model = joblib.load('model.pkl')

# Caricamento test set
test = pd.read_csv('test.csv')

# Preprocessing coerente con training
# (stessi passaggi, stessi parametri)

# Predizione
predictions = model.predict(test)

# Creazione submission
submission = pd.DataFrame({
    'PassengerId': test['PassengerId'],
    'Target': predictions
})

submission.to_csv('submission.csv', index=False)
```

17.2 Best Practice per Kaggle

1. **Consistenza preprocessing**: Applica gli stessi passaggi di training al test set
2. **Allineamento colonne**: Assicurati che le colonne siano nello stesso ordine
3. **Cross-validation locale**: Simula la submission con CV per stimare il public LB score
4. **Ensemble di modelli**: Combina predizioni di modelli diversi
5. **Calibrazione**: Calibra le probabilità se necessario (Platt scaling, isotonic regression)

17.3 Gestione Colonne Mancanti

```
# Allineamento colonne test/train
X_test = X_test.reindex(columns=train_columns, fill_value=0)

# Gestione nuove categorie nel test set
for col in categorical_features:
    X_test[col] = X_test[col].cat.add_categories(['NewCategory']).fillna('NewCategory')
```

18. Best Practices e Errori Comuni

18.1 Best Practices Generali

1. **EDA approfondita**: Capire i dati prima di modellare
2. **Baseline semplice**: Inizia con un modello semplice (Linear Regression, Logistic Regression)
3. **Train/Validation/Test**: Usa almeno 3 split per valutazione robusta
4. **Cross-validation**: Sempre usare CV per tuning e valutazione
5. **Feature engineering iterativo**: Aggiungi feature gradualmente, valuta l'impatto
6. **Salva tutto**: Modelli, preprocessing, parametri, metriche
7. **Reproducibilità**: Fissa random_state, documenta tutti i passaggi
8. **Interpretabilità**: Usa SHAP/feature importance per capire il modello

18.2 Errori Comuni da Evitare

1. **Data leakage**: Usare informazioni dal test set durante il training
    - **Soluzione**: Fit preprocessing solo sul training set
2. **Normalizzare prima dello split**: Scalare tutto il dataset insieme
    - **Soluzione**: Split prima, poi scalare separatamente
3. **One-Hot Encoding inconsistente**: Categorie diverse tra train e test
    - **Soluzione**: Fit encoder sul training, trasformare test con lo stesso encoder
4. **Valutare solo accuracy**: In dataset sbilanciati, accuracy è fuorviante
    - **Soluzione**: Usa precision, recall, F1, ROC AUC
5. **Overfitting al validation set**: Tuning eccessivo sul validation set
    - **Soluzione**: Usa nested CV o tiene un test set completamente separato
6. **Ignorare missing values**: Trattare tutti i missing come zero o rimuoverli
    - **Soluzione**: Analizza pattern di missing, usa strategie appropriate
7. **Feature engineering su tutto il dataset**: Creare feature usando statistiche globali
    - **Soluzione**: Calcola statistiche solo sul training set, applica al test

18.3 Takeaways per la Revisione

Concetti Fondamentali:

- **Regressione Lineare**: $y = \beta_0 + \sum \beta_i x_i + \epsilon$ con MSE come loss
- **Logistic Regression**: Sigmoide per probabilità, decision boundary a 0.5
- **Outlier**: Z-score (assume normalità) o IQR (robusto)
- **Train/Test Split**: Sempre stratificato per classificazione

Metriche di Valutazione:

- **Regressione**: MSE, RMSE, R^2
- **Classificazione**: Accuracy, Precision, Recall, F1, ROC AUC, Matrice di Confusione

Ensemble Learning:

- **Bagging (Random Forest)**: Modelli paralleli, riduce variance
- **Boosting (XGBoost)**: Modelli sequenziali, corregge errori, potente ma sensibile

Regularizzazione:

- **Per alberi**: max_depth, min_samples_leaf, subsample
- **Per reti**: Dropout, L1/L2, early stopping

Best Practices:

- **Sempre**: Fit preprocessing sul training, trasformare test
- **Feature engineering**: Iterativo, documentato, riproducibile
- **Tuning**: Grid/Random Search o Optuna con cross-validation

Conclusione

Questa guida copre l'intero spettro di concetti di Machine Learning trattati nei materiali del repository, dai fondamenti della regressione lineare fino alle tecniche avanzate di ensemble learning e interpretazione dei modelli.

Punti chiave da ricordare:

1. **Comprendere i dati** è il primo e più importante passo
2. **Preprocessing corretto** è essenziale per evitare data leakage
3. **Valutazione robusta** richiede cross-validation e metriche appropriate
4. **Feature engineering** è spesso più importante della scelta dell'algoritmo
5. **Ensemble methods** (Random Forest, XGBoost) sono tra i più potenti
6. **Interpretabilità** è cruciale per fiducia e miglioramento del modello

7. **Regularizzazione** previene overfitting e migliora generalizzazione

Il percorso di apprendimento dovrebbe seguire questa progressione:

1. Regressione Lineare → Capire i fondamenti
2. Logistic Regression → Introduzione alla classificazione
3. EDA e Feature Engineering → Preparazione dati
4. Decision Trees → Modelli non lineari
5. Random Forest → Bagging e ensemble
6. XGBoost → Boosting avanzato
7. Cross-Validation e Tuning → Valutazione robusta
8. SHAP e Interpretabilità → Comprensione del modello

Buono studio!